Fractal Continual Learning:
Recursive Structural Growth, Long-Horizon Adaptation, and the Boundary of Structural Advantage

HUSHFUSION Forge Research

October 2026

Abstract

Continual learning systems update parameters under a fixed computational graph; whether the graph itself should grow is rarely posed as a testable question. We ask it directly. We introduce a recursive fractal-loop learner in which a dense root loop and sparse descendant loops communicate through normalized convex gates, and a relative-decay signal grows descendants when a structure's improvement rate decays relative to its own early rate. In a non-stationary control environment assembled from a two-flow-ring μ-dynamics model, a paired 72,335-round run shows both the fractal tree and a fixed single-node learner keep learning: warm control cost falls from roughly 130–176 steps to roughly 9 steps (cold random baseline, $\sim$32 steps; a 3.5$\times$ gain). The tree grows to seven nodes and stabilizes. The structural advantage does not: after convergence, fractal warm cost is indistinguishable from fixed (mean difference $<$0.3 steps; exceedance 5.8% vs. 5.9%), a result consistent across three seeds and 701 real configuration–score records. We therefore report not a universal benefit of fractal growth but a reproducible boundary: recursive structural plasticity is implementable and stable over long horizons, yet confers no measurable advantage when marginal returns are weak. The protocol separates "structural plasticity" from "structural advantage" so that a failure of the latter does not erase evidence for the former.

1 Introduction

Continual learning is normally stated as a weight-update problem: keep the graph fixed and adapt parameters so that new tasks are learned without forgetting old ones [1, 3, 2]. This framing leaves an unasked question: when the task itself drifts, should the *organization of the updating loops* change, not just the weights inside them? The question matters for long-running controllers, where a learner must track a moving optimum for tens of thousands of rounds while preserving a stable response. Yet whether structural growth helps is nearly always asserted, not measured.

We treat it as a measurement problem and make three claims, each with a separate evidence chain. First, a recursive learner can grow descendant loops from a single root online, communicate across levels through a normalized convex gate, and stabilize rather than grow without bound. Second, this mechanism can keep learning for 72,335 rounds under a drifting control condition, matching a fixed learner in sustained adaptation. Third—the claim that defines the paper's value—the grown structure gives no measurable performance advantage in this regime: after convergence, paired fractal and fixed warm costs differ by less than 0.3 steps, with identical exceedance rates; three-seed and 701-sample real-data checks agree. The outcome is a boundary result. It shows where structural plasticity is useful (as a model of long-horizon adaptation) and where it is not (as a performance lever when marginal returns are weak). We release the protocol, the negative results, and the failure analyses so that the boundary is reproducible.

Table 1: Long-horizon paired outcome (72,335 rounds). Warm rows are recommenders; cold rows are random controls. Exceedance = crossing the safety ceiling μ_{ceil}.

Metric	Fixed	Fractal
Warm mean steps (full run)	11.1	11.5
Warm mean steps (converged, round $\geq$2000)	$\sim$9	$\sim$9
Cold mean steps	31.6	31.9
Exceedance rate	5.9%	5.8%
Final tree nodes	1	7

2 Results

2.1 Long-horizon continual adaptation succeeds in both structures

Two learners, a fixed single-node network and a recursively grown fractal tree, were paired at every round on the same drifting initial condition μ_0 of a deterministic μ-dynamics controller (protocol in Methods). Each selected a control point from its accumulated experience and executed the environment.

Both structures learned and kept learning. Warm performance fell from roughly 130–176 steps in the first 1,000 rounds to roughly 9 steps after 2,000 rounds, and stayed there for the remaining 70,000 rounds; the cold random baseline remained near 32 steps throughout (Table 1). Continual adaptation was therefore effective and stable, and it did not decay: this is the first result, and it holds for both architectures.

2.2 Structure grows, then stabilizes

The fractal tree grew six times (rounds 46, 57, 73, 80, 87, 94), from one to seven nodes, and then stopped. Growth was driven by a relative-decay detector (Methods): a descendant spawns when the exponentially weighted improvement rate of validation loss falls below a fraction of that structure's early improvement rate. After reaching seven nodes the detector never fired again for the remaining 72,000 rounds. Unbounded growth is not observed; the tree settles.

2.3 The structural advantage does not appear

Growth worked; advantage did not. After convergence, fractal warm cost was statistically indistinguishable from fixed: paired outcomes were fixed-winner 16.1%, fractal-winner 20.9%, and ties 63.0%, yet the full-run means were 11.5 vs. 11.1 steps—the win-rate signal and the mean contradict, which is the signature of noise rather than a real edge. Exceedance matched (5.8% vs. 5.9%). Two corroborating checks agree. A three-seed short-run comparison gave one seed favoring fractal, one tied, one favoring fixed. A clean five-fold cross-validation on 701 real Forge configuration–score records gave fixed 0.0719 ± 0.0468 and a three-descendant fractal tree 0.0861 ± 0.0536 mean squared error—no fractal advantage there either. In this regime the additional six nodes were a complexity tax, not a lever.

3 Related Work

Our design sits at the meeting point of optimization-based meta-learning, which separates fast task adaptation from slower meta-parameter updates [1, 3]; continual-learning consolidation, which

addresses forgetting under sequential tasks [2]; and fractal allocation arguments, which suggest that when marginal returns decline with depth, computation need not be spread evenly across levels—a principle with roots in Murray's minimum-work analysis of branching networks [6] and formalized in our companion Lean development [5]. We keep these as motives, not predictions: none of them implies that a small learner on a 701-record task will benefit from fractal organization, and our result confirms that.

4 Discussion

Two statements must be held apart. "Recursive structural plasticity is implementable and stable" is true: growth events fire, communication through convex gates works, and the system is trainable for 72,000 rounds. "Structural growth improves performance" is false in the regime studied; the grown nodes added cost without measurable return. Conflating the two is the standard error in this line of work, and separating them is the paper's methodological point.

Why did the advantage fail to appear? The prefix-concentration argument that motivates fractal allocation [5] requires diminishing returns strong enough that concentrating budget in early levels beats spreading it. On a small learner with limited data, marginal returns do not decline sharply; splitting the budget across specialists merely dilutes each specialist's evidence. The hypothesis is not refuted in general—it is bounded. The natural next tests are larger networks, multi-domain streams, and tasks with genuinely multi-scale structure, where diminishing returns are expected and where a fractal organization should, if the theory holds, earn its complexity.

The practical lesson is procedural. Evaluate structural growth in three stages: (i) does the graph mutate safely, (ii) does the learner keep adapting after mutation, and (iii) does the mutation improve a pre-registered metric under matched compute and data? This paper ran all three and reports each answer separately.

5 Methods

5.1 Recursive fractal-loop architecture

Each node v holds a predictor $f_v(x;\theta_v)$ and may have descendants $\mathcal{C}(v)$. The node output is a convex combination,

$$y_v(x) = g_{v,0}\, f_v(x;\theta_v) + \sum_{u\in\mathcal{C}(v)} g_{v,u}\, y_u(x), \qquad g_{v,j}\geq 0, \quad \sum_j g_{v,j} = 1, \tag{1}$$

with gates produced by a softmax. Descendants are not trusted immediately: they are warmed independently before entering the parent's mixture, which prevents the "dead-child" failure where a zero-initialized gate gives a child zero contribution and therefore zero gradient.

The root is updated every round; a node at depth d updates at frequency D^{-d} for sparse factor D (we use $D = 4$). This concentrates computation in the shallow prefix—the operational analogue of a fractal allocation rule.

5.2 Growth rule

Growth is triggered by relative decay of validation loss. Let $\bar{\imath}_t$ be an exponentially weighted improvement rate and $\imath_0$ the structure's early average improvement rate. A descendant is spawned when $\bar{\imath}_t/\imath_0$ stays below a threshold (0.2) for several consecutive observations; the detector resets after

growth. A clean-loss filter excludes safety-penalty values, because a penalty spike is not evidence of a structural bottleneck. We include the failure that motivated the design: an initial absolute rule ("loss no longer decreases") never fires, because a shallow model can improve forever at a slow rate.

5.3 Control environment

We assemble a deterministic μ-dynamics controller from the two-flow-ring model:

$$\mu_{t+1} = \mu_t + \eta_t(1 - \mu_t), \qquad \eta_t = \eta_{\text{ext}} + (2\lambda - \lambda^2)(1 - \mu_t), \tag{2}$$

where η_{ext} is an external drive and λ a convergence parameter for a candidate self-flattening term. The task is to reach $\mu \geq 0.999$ (work window) without crossing $\mu_{\text{ceil}} = 1 - m_e/m_i \approx 0.999780568$ (safety ceiling). Cost is the number of steps to the window; ceiling crossing is a failure. The environment is deterministic, has a clear safety boundary, and is non-stationary because a fresh $\mu_0 \sim U(0.01, 0.8)$ is drawn every round. It is a benchmark for method study, not evidence about any experimental antigravity device; λ remains an input gap in the upstream model.

5.4 Paired long-horizon protocol

Every round, both learners see the same μ_0. Each selects a control point from its experience (UCB over a 20×10 grid of $(\eta_{\text{ext}}, \lambda)$), executes the environment, and updates its parameters from a sliding window of recent experience (capacity 400). Odd rounds are cold random controls, even rounds are warm recommendations. The deployed run used 72,335 rounds, independent checkpoints every 10 rounds, and a supervisor that restarted the daemon on failure. The fractal tree was allowed to grow without a preselected depth, subject to a safety cap of seven nodes.

5.5 Failure analyses that shaped the design

Four failures are worth recording. (i) An absolute "no-improvement" growth detector never fires on shallow networks (they improve slowly forever); the relative-decay detector fixed this. (ii) Zero-initialized child gates produce dead children; uniform gate initialization plus a warm-up phase fixed this. (iii) Growing a layer without reinitializing per-parameter step-size meta-parameters damaged the fast-adaptation loop, so structural growth and meta-parameter growth cannot be treated as independent. (iv) A standard RSI indicator is inapplicable to a monotonically improving loss trajectory (one-sided deltas give RSI $\equiv$ 100), so the term is used here for "recursive self-improvement," not the financial indicator.

Data and code availability

Implementation, supervisors, the 72,335-round paired trace, and analysis scripts are in the HUSH-FUSION Forge repository [4]. Writing standards for this paper are in `WRITING-STANDARD.md`.

References

[1] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1126–1135, 2017.

[2] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[3] Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. Meta-SGD: Learning to learn quickly for few-shot learning. *arXiv preprint arXiv:1707.09835*, 2017.

[4] Yuanjie Liu. HUSHFUSION Forge: design search, knowledge reuse, and continual-learning daemons. `https://github.com/logos-42/hushfusion-forge`, 2026. Long-horizon fracta-loop trace and reproduce scripts.

[5] Yuanjie Liu. LmPrinciple: Fractal allocation theorems for diminishing-return structures. `https://github.com/logos-42/lm-principle`, 2026. Lean formalization: Fractal.lean, prefix_allocation_optimal, fractal_beats_uniform.

[6] Cecil D. Murray. The physiological principle of minimum work: I. The vascular system and the cost of blood volume. *Proceedings of the National Academy of Sciences*, 12(3):207–214, 1926.